

Exp6

```
import java.sql.*;

public class DatabaseCheckpointCustomCheck {

    static final String DB_URL = "jdbc:sqlite:student.db";

    public static void setupDatabase() throws SQLException {

        try (Connection conn = DriverManager.getConnection(DB_URL);

            Statement stmt = conn.createStatement()) {

            stmt.execute("CREATE TABLE IF NOT EXISTS students (" +

                "id INTEGER PRIMARY KEY, name TEXT, marks INTEGER)");

            stmt.execute("DELETE FROM students");

            stmt.execute("INSERT INTO students VALUES (1, 'Amit', 78)");

            stmt.execute("INSERT INTO students VALUES (2, 'Neha', 45)");

            stmt.execute("INSERT INTO students VALUES (3, 'Ravi', 92)");

        }

    }

    public static boolean customCheckpoint(int studentId,

        String expectedStatus)

        throws SQLException {

        try (Connection conn = DriverManager.getConnection(DB_URL);

            PreparedStatement ps = conn.prepareStatement(

                "SELECT marks FROM students WHERE id = ?")) {

            ps.setInt(1, studentId);

            ResultSet rs = ps.executeQuery();

            if (!rs.next()) {

                System.out.println("FAIL: No record found for id " + studentId);

                return false;

            }

            int marks = rs.getInt("marks");

            String actualStatus = (marks >= 50) ? "Pass" : "Fail";

            Software Testing Methodologies Lab | 7

            if (actualStatus.equals(expectedStatus)) {
```

```

System.out.println("PASS: Student " + studentId +
" status = " + actualStatus +
" (Custom check matched)");
return true;
} else {
System.out.println("FAIL: Student " + studentId +
" expected " + expectedStatus +
", got " + actualStatus);
return false;
}
}
}

public static void main(String[] args) throws Exception {
setupDatabase();
customCheckpoint(1, "Pass");
customCheckpoint(2, "Fail");
customCheckpoint(3, "Pass");
}
}

javac -cp sqlite-jdbc-3.34.0.jar DatabaseCheckpointCustomCheck.java
java -cp ".;sqlite-jdbc-3.34.0.jar" DatabaseCheckpointCustomCheck
Exp7a

```

```

import java.util.Random;

public class DynamicDataTest {

public static double calculateDiscount(double price, String category) {
switch (category) {
case "electronics": return price * 0.9;
case "clothing": return price * 0.8;
default: return price;
}
}
}

```

```

public static void main(String[] args) {
    String[] categories = {"electronics", "clothing", "grocery"};
    Random rand = new Random();
    System.out.println("---- Dynamic Test Data Submission ----");
    for (int i = 1; i <= 5; i++) {
        int price = 100 + rand.nextInt(901);
        String category = categories[rand.nextInt(categories.length)];
        double result = calculateDiscount(price, category);
        System.out.printf(
            "Test %d: price=%d, category=%s -> discounted=%.2f%n",
            i, price, category, result);
    }
}

```

Exp7b

```

import java.io.*;

public class FlatFileTest {

    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createFlatFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("testdata.txt"))) {
            pw.println("500,electronics");
            pw.println("300,clothing");
            pw.println("150,grocery");
        }
    }
}

```

```

public static void flatFileTest() throws IOException {
    System.out.println("---- Data Driven Test via Flat File ----");
    try (BufferedReader br = new BufferedReader(
        new FileReader("testdata.txt"))) {
        String line;
        while ((line = br.readLine()) != null) {
            String[] parts = line.split(",");
            double price = Double.parseDouble(parts[0]);
            String category = parts[1];
            double result = calculateDiscount(price, category);
            System.out.printf(
                "price=%.1f, category=%s -> discounted=%.2f%n",
                price, category, result);
        }
    }
}

public static void main(String[] args) throws IOException {
    createFlatFile();
    flatFileTest();
}
}

```

Exp7c

```

import javax.swing.*.*;
import javax.swing.table.DefaultTableModel;

public class FrontEndGridTest {

    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }
}

```

```

}
}
public static void main(String[] args) {
    JFrame frame = new JFrame("Front End Grid - Data Driven Test");
    String[] columns = {"Price", "Category", "Result"};
    DefaultTableModel model = new DefaultTableModel(columns, 0);
    Object[][] gridData = {
        {500.0, "electronics"},
        {300.0, "clothing"},
        {150.0, "grocery"}
    };
    for (Object[] row : gridData) {
        double price = (double) row[0];
        String category = (String) row[1];
        double result = calculateDiscount(price, category);
        model.addRow(new Object[]{
            price, category, String.format("%.2f", result)
        });
    }
    JTable table = new JTable(model);
    frame.add(new JScrollPane(table));
    frame.setSize(400, 200);
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    frame.setVisible(true);
}
}

```

Exp7e

```

public class BatchNoParams {
    public static void testAddition() {
        assert 2 + 3 == 5;
        System.out.println("test_addition: PASS");
    }
}

```

```
}
```

Software Testing Methodologies Lab | 11

```
public static void testSubtraction() {  
    assert 5 - 2 == 3;  
    System.out.println("test_subtraction: PASS");  
}  
  
public static void main(String[] args) {  
    System.out.println("---- Batch Testing (No Parameters) ----");  
    testAddition();  
    testSubtraction();  
}  
}
```

Exp7f

```
public class BatchWithParams {  
    static void testOperation(int a, int b, char op, int expected) {  
        int result = 0;  
        switch (op) {  
            case '+': result = a + b; break;  
            case '-': result = a - b; break;  
            case '*': result = a * b; break;  
        }  
        String status = (result == expected) ? "PASS" : "FAIL";  
        System.out.printf(  
            "Test: %d %c %d = %d (expected %d) -> %s%n",  
            a, op, b, result, expected, status);  
    }  
  
    public static void main(String[] args) {  
        int[][] testCases = {  
            {2, 3, '+', 5},  
            {10, 4, '-', 6},  
            {3, 3, '*', 9},  
        }  
    }  
}
```

```

{5, 5, '-', 1}
};

System.out.println("---- Batch Testing (With Parameters) ----");

for (int[] tc : testCases) {
    testOperation(tc[0], tc[1], (char) tc[2], tc[3]);
}
}
}

Exp8

import java.io.*;

public class DataDrivenBatch {

    public static double calculateDiscount(double price, String category) {
        switch (category) {
            case "electronics": return price * 0.9;
            case "clothing": return price * 0.8;
            default: return price;
        }
    }

    public static void createCsvFile() throws IOException {
        try (PrintWriter pw = new PrintWriter(new FileWriter("batch_data.csv"))) {
            pw.println("Price,Category,ExpectedResult");
            pw.println("500,electronics,450");
            pw.println("300,clothing,240");
            pw.println("150,grocery,150");
            pw.println("1000,electronics,900");
        }
    }

    public static void dataDrivenBatch() throws IOException {
        System.out.println("---- Data Driven Batch Test ----");

        int passCount = 0, failCount = 0;

        try (BufferedReader br =

```

```

new BufferedReader(new FileReader("batch_data.csv"))) {
String line = br.readLine();
while ((line = br.readLine()) != null) {
String[] parts = line.split(",");
double price = Double.parseDouble(parts[0]);
String category = parts[1];
double expected = Double.parseDouble(parts[2]);
double actual = calculateDiscount(price, category);
if (actual == expected) {
System.out.printf(
"PASS: %s, price=%.1f -> %.1f%n",
category, price, actual);
passCount++;
} else {
System.out.printf(
"FAIL: %s, price=%.1f -> got %.1f, expected %.1f%n",
category, price, actual, expected);
failCount++;
}
}
}

System.out.println("\nBatch Summary: " + passCount +
" Passed, " + failCount + " Failed");
}

public static void main(String[] args) throws IOException {
createCsvFile();
dataDrivenBatch();
}
}

Exp9
import java.util.logging.*;

```



```

import java.io.IOException;

public class SilentModeTest {

    static Logger logger = Logger.getLogger("SilentTestLogger");

    static void setupLogger() throws IOException {

        logger.setUseParentHandlers(false);

        FileHandler fh = new FileHandler("silent_test_log.txt");

        fh.setFormatter(new SimpleFormatter());

        logger.addHandler(fh);

    }

    static void testCase(String name, Object actual, Object expected) {

        try {

            if (actual.equals(expected)) {

                logger.info(name + ": PASS (actual=" + actual +

                    ", expected=" + expected + ")");

            } else {

                logger.severe(name + ": FAIL (actual=" + actual +

                    ", expected=" + expected + ")");

            }

        } catch (Exception e) {

            logger.severe(name + ": ERROR - " + e.getMessage());

        }

    }

    public static void main(String[] args) throws IOException {

        setupLogger();

        testCase("Addition Test", 2 + 2, 4);

        testCase("Subtraction Test", 10 - 5, 5);

        testCase("Multiplication Test", 3 * 3, 10);

        testCase("String Test", "abc".toUpperCase(), "ABC");

        System.out.println(

            "Batch execution completed silently. " +

            "Check silent_test_log.txt for results.");
    }
}

```

```
}  
}
```

Exp10

```
import java.awt.*;  
import java.awt.event.KeyEvent;  
import java.awt.image.BufferedImage;  
import javax.imageio.ImageIO;  
import java.io.File;  
  
public class CalculatorTest {  
    public static void main(String[] args) throws Exception {  
        Runtime.getRuntime().exec("calc.exe");  
        Thread.sleep(2000);  
        Robot robot = new Robot();  
        robot.setAutoDelay(200);  
        typeKey(robot, KeyEvent.VK_7);  
        typeKey(robot, KeyEvent.VK_ADD);  
        typeKey(robot, KeyEvent.VK_5);  
        typeKey(robot, KeyEvent.VK_EQUALS);  
        Thread.sleep(1000);  
        Rectangle screenRect = new Rectangle(  
            Toolkit.getDefaultToolkit().getScreenSize());  
        BufferedImage capture =  
            robot.createScreenCapture(screenRect);  
        ImageIO.write(capture, "png",  
            new File("calculator_result.png"));  
        System.out.println("Test executed: 7 + 5 =");  
        System.out.println(  
            "Screenshot saved as calculator_result.png -- " +  
            "compare visually to confirm result is 12");  
    }  
  
    static void typeKey(Robot robot, int keyCode) {
```

```
robot.keyPress(keyCode);  
robot.keyRelease(keyCode);  
}  
}
```